

SECTION 30 — FRAMEWORK CODE SNIPPET LIBRARY (Refined)

Topics Covered

- 30.1-30.26 (26 headers)

First pass — 3 parallel batch subagents

30.1 Selenium Driver Factory — Refined

Theory: Centralizes WebDriver creation behind a single Factory so tests never call `new ChromeDriver()` directly. Reads `browser` + `target` (local/remote) config, builds options, handles Selenium Manager or Grid URL, fails fast on unsupported browsers. Keeps creation stateless; returns `WebDriver` abstraction; leaves lifecycle (quit) to `BaseTest` / `DriverManager`. Essential for cross-browser maintainability — one place to add Edge/Safari/Grid. Enterprise Relevance: Factory is the seam where local/CI/Grid browsers diverge without touching tests. New browser support = 5 lines in factory, zero test changes.

```
public final class DriverFactory {
    private DriverFactory() {}
    public static WebDriver create(String browser, boolean headless, String gridUrl)
    throws Exception {
        ChromeOptions o = new ChromeOptions();
        if (headless) o.addArguments("--headless=new", "--no-sandbox", "--window-
size=1920,1080");
        if (gridUrl != null) return new RemoteWebDriver(new URI(gridUrl).toURL(), o);
        return switch (browser.toLowerCase()) {
            case "chrome" -> new ChromeDriver(o);
            case "firefox" -> new FirefoxDriver(new FirefoxOptions());
            case "edge" -> new EdgeDriver(new EdgeOptions());
            default -> throw new IllegalArgumentException("Unsupported browser: " +
browser);
        };
    }
}
```

Triage: `SessionNotCreated` → version skew (Manager/CfT), not factory; unsupported-browser error → config typo. Anti: `new ChromeDriver()` in tests; options scattered per test; Grid URL hardcoded. Qs: Factory vs direct construction — what breaks without it?

30.2 ThreadLocal Driver — Refined

Theory: Provides one isolated `WebDriver` per TestNG worker thread for safe parallel execution. TestNG runs `@BeforeMethod` → `@Test` → `@AfterMethod` on the same thread, so `ThreadLocal.get()` always returns the correct session. Holder must be `static final`; explicit `set()` in setup; always `quit()` + `remove()` in teardown (pooled-thread reuse leaks stale sessions without remove). Null-guard with thread name for clear diagnostics. Enterprise Relevance: The parallel-safety primitive for

Selenium — Sec 22.12 theory applied as copy-paste snippet. Missing `remove()` exhausts Grid slots mid-suite (the classic "passes with 2 threads, dies with 8").

```
public final class DriverManager {
    private static final ThreadLocal<WebDriver> DRIVER = new ThreadLocal<>();
    private DriverManager() {}
    public static void set(WebDriver d) { DRIVER.set(d); }
    public static WebDriver get() {
        if (DRIVER.get() == null)
            throw new IllegalStateException("No driver on " +
Thread.currentThread().getName());
        return DRIVER.get();
    }
    public static void quit() { try { if (DRIVER.get() != null) DRIVER.get().quit(); }
finally { DRIVER.remove(); } }
}
```

Triage: cross-test navigation → static driver (migrate here); slot exhaustion → missing remove; null guard message names the thread. Anti: static WebDriver, set-without-remove, storing ThreadLocal in pages. Qs: Why `static final` holder + per-thread values? What happens without `remove()` ?

30.3 Base Test — Refined

Theory: Owns common TestNG lifecycle: create driver in `@BeforeMethod` (fresh per test = isolation), navigate to base URL, maximize, configure timeouts; capture screenshot on failure + quit in `@AfterMethod(alwaysRun=true)` . Exposes `getDriver()` for pages and listeners. Keeps tests DRY, enforces isolation with fresh driver per method, integrates config + ThreadLocal cleanup + reporting in one Deloitte-standard superclass. Enterprise Relevance: Every test inherits correctness (isolation, cleanup, evidence) — juniors write test logic only. Fresh-driver-per-method trades minutes (browser restart) for zero cross-test contamination.

```
public abstract class BaseTest {
    protected WebDriver driver;
    @BeforeMethod @Parameters("browser")
    public void setUp(@Optional("chrome") String browser) throws Exception {
        WebDriver d = DriverFactory.create(browser, Config.headless(),
Config.gridUrl());
        DriverManager.set(d); driver = d;
        driver.manage().window().maximize();
        driver.get(Config.baseUrl());
    }
    @AfterMethod(alwaysRun = true)
    public void tearDown(ITestResult r) {
        if (!r.isSuccess()) ScreenshotUtil.capture(driver, r.getName());
        DriverManager.quit();
    }
    public WebDriver getDriver() { return driver; }
}
```

Triage: cross-test pollution → shared/class-level driver (move to method); orphan browsers → missing `alwaysRun`; slow suite → class-level driver tradeoff (documented, risky). Anti: logic in `BaseTest` (belongs in `pages/utils`); swallowing setup errors. Qs: Method-level vs class-level driver trade-off?

30.4 Base Page — Refined

Theory: Abstract parent for all Page Objects holding `WebDriver` + `WebDriverWait`. Provides reusable `click()`, `type()`, `getText()` wrappers with logging + wait handling. Prevents static drivers (constructor injection of thread-local driver required), centralizes exception handling (wrap to `FrameworkException` with locator + URL), ensures consistent synchronization + auditability across POM suites. Note: prefer `By` locators + explicit waits over `PageFactory.initElements` proxies (Sec 15.2) — modernized snippet below uses `By`. Enterprise Relevance: Every page inherits wait discipline + error context — no raw `findElement` in page methods, no `Thread.sleep` anywhere.

```
public abstract class BasePage {
    protected final WebDriver driver;
    protected final WebDriverWait wait;
    protected final Logger log = LoggerFactory.getLogger(getClass());
    protected BasePage(WebDriver driver) {
        this.driver = Objects.requireNonNull(driver);
        this.wait = new WebDriverWait(driver, Duration.ofSeconds(10));
    }
    protected void click(By loc) {
        try { wait.until(ExpectedConditions.elementToBeClickable(loc)).click(); }
        catch (TimeoutException e) { throw new FrameworkException("TIMEOUT-01", "Click "
+ loc, e); }
    }
    protected void type(By loc, String text) {
        WebElement e = wait.until(ExpectedConditions.visibilityOfElementLocated(loc));
        e.clear(); e.sendKeys(text);
    }
    protected String text(By loc) { return
wait.until(ExpectedConditions.visibilityOfElementLocated(loc)).getText(); }
}
```

Triage: `NoSuch` across pages → locator strategy (not waits); inconsistent timeouts → centralize here. Anti: `PageFactory.initElements` proxies (stale-prone), public `WebElements`, assertions in pages (except `isLoading`). Qs: `By` + waits vs `PageFactory` proxies?

30.5 Explicit Wait Utility — Refined

Theory: Replaces flaky `Thread.sleep()` and implicit waits with `WebDriverWait` + `ExpectedConditions`. Central `WaitUtils` offers `forVisible()`, `forClickable()`, `forInvisible()`, plus custom `FluentWait` ignoring `StaleElementReferenceException`. Selenium 4 `Duration`-based API; configurable timeout/polling; clear timeout messages with locator. Critical for dynamic apps (AJAX, spinners, overlays). Enterprise Relevance: Centralized waits = one timeout policy, one failure format, zero sleeps. Every `Thread.sleep` in review gets replaced with the matching util.

```

public final class WaitUtils {
    private WaitUtils() {}
    public static WebElement visible(WebDriver d, By loc, int secs) {
        return new WebDriverWait(d, Duration.ofSeconds(s))
            .withMessage("Not visible: " + loc)
            .until(ExpectedConditions.visibilityOfElementLocated(loc));
    }
    public static WebElement clickable(WebDriver d, By loc, int secs) {
        return new WebDriverWait(d, Duration.ofSeconds(secs))
            .ignoring(StaleElementReferenceException.class)
            .until(ExpectedConditions.elementToBeClickable(loc));
    }
    public static boolean invisible(WebDriver d, By loc, int secs) {
        return new WebDriverWait(d, Duration.ofSeconds(secs))
            .until(ExpectedConditions.invisibilityOfElementLocated(loc));
    }
}

```

Triage: `TimeoutException` message names the locator + condition (no guessing). Anti: `Thread.sleep`, implicit waits, inline `new WebDriverWait` with magic timeouts. Qs: What three parameters does every wait need?

30.6 Screenshot Utility — Refined

Theory: Standardizes evidence capture via `TakesScreenshot.getScreenshotAs()`: full-page `FILE`, `BYTES` for reports, `BASE64` for HTML/Allure embedding, `WebElement` screenshots after scroll-into-view. Names `TestName_timestamp.png` under `target/screenshots`; called from `ITestListener.onTestFailure()` before `quit()` (after quit, no browser to shoot). Handles `Augmenter` for remote drivers + null-driver guards for audit trails. Enterprise Relevance: Screenshots are release-gate evidence — every failed gate test links one. `BASE64` embeds directly in Allure/Extent (no file-path breakage in CI artifacts).

```

public final class ScreenshotUtil {
    private ScreenshotUtil() {}
    public static String capture(WebDriver driver, String name) {
        if (driver == null) return null;
        String path = "target/screenshots/" + name + "_" + System.currentTimeMillis() + ".png";
        try {
            FileHandler.copy(((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE), new File(path));
        } catch (IOException e) { throw new FrameworkException("SHOT-01", "Screenshot failed", e); }
        return path;
    }
    public static String base64(WebDriver driver) {
        return ((TakesScreenshot) driver).getScreenshotAs(OutputType.BASE64);
    }
}

```

Triage: black screenshots → headless viewport/size; quit before capture → order (capture then quit); remote fails → Augmenter. Anti: on-demand only; PNGs in Git; no timestamp (overwrites). Qs: FILE vs BYTES vs BASE64 — when each?

30.7 Browser Options — Refined

Theory: Encapsulates `ChromeOptions` / `FirefoxOptions` / `EdgeOptions` : headless, incognito/private, window size, disable notifications/infobars, download directory, proxy, Grid capabilities. Built by factory from system properties or `config.properties` with safe defaults. Selenium Manager resolves binaries automatically since 4.6. Keeps environment-specific tweaks out of tests for clean CI execution. Enterprise Relevance: Options centralization = CI/local/headless matrix without test changes. Download dir, proxy, and headless flags are the top three CI-specific needs.

```
public static ChromeOptions chromeOptions(boolean headless) {
    ChromeOptions o = new ChromeOptions();
    o.addArguments("--start-maximized", "--disable-notifications", "--disable-
infobars");
    if (headless) o.addArguments("--headless=new", "--window-size=1920,1080");
    o.setPageLoadStrategy(PageLoadStrategy.NORMAL);
    Map<String, Object> prefs = Map.of("download.default_directory", "/tmp/dl");
    o.setExperimentalOption("prefs", prefs);
    return o;
}
```

Triage: download prompts block CI → prefs dir; notifications overlay → disable; headless size 800x600 screenshots → window-size. Anti: options in tests; `--headless` legacy flag; infobars breaking locators. Qs: Which options differ local vs CI?

30.8 Window / Frame Handling — Refined

Theory: Manages multiple windows/tabs via `getWindowHandles()` + `switchTo().window(handle)` (save parent, click, switch, validate, close children); iframes via `switchTo().frame(index|name|element)` + `defaultContent()`, always waiting for frame availability. Wraps in utilities returning to default context in `finally` (avoids stuck driver state in complex portals). Enterprise Relevance: Window/frame bugs strand the driver in wrong context — every subsequent test fails mysteriously. `finally` -guarded utilities make context leaks impossible.

```
public static void switchToNewWindow(WebDriver d, String parent) {
    new WebDriverWait(d, Duration.ofSeconds(10)).until(x ->
x.getWindowHandles().size() > 1);
    for (String h : d.getWindowHandles()) if (!h.equals(parent)) {
        d.switchTo().window(h); break; }
    }
    public static void toFrame(WebDriver d, By loc) {
        try {
            new WebDriverWait(d, Duration.ofSeconds(10))
                .until(ExpectedConditions.frameToBeAvailableAndSwitchToIt(loc));
        } finally { /* caller must defaultContent in its finally */ }
    }
}
```

Triage: NoSuch in frame → wrong context (switch first); stuck after test → missing defaultContent/close in finally. Anti: index-only frame switching; no parent-handle save; assuming window order. Qs: How do you guarantee return to default context?

30.9 DataProvider — Refined

Theory: Enables data-driven TestNG tests by feeding multiple datasets to one `@Test` method — separating test logic from test data. Supports inline `Object[][]`, external CSV/Excel/JSON readers, `parallel=true` for concurrent execution, `ITestContext` filtering. Pairs with method-parallel `ThreadLocal` drivers for isolated regression coverage without duplicating test methods. Enterprise Relevance: Data-driven tests multiply coverage per method (1 method × 50 rows = 50 cases). Parallel DataProviders + `ThreadLocal` drivers = data-volume testing at speed.

```
@DataProvider(name = "login", parallel = true)
public Object[][] data() {
    return new Object[][] { {"admin", "admin123", true}, {"locked", "pass", false} };
}
@Test(dataProvider = "login")
public void loginTest(String user, String pwd, boolean valid) {
    new LoginPage(DriverManager.get()).loginAs(user, pwd);
    Assertions.assertEquals(valid, new HomePage(DriverManager.get()).isLoaded());
}
```

Triage: parallel DataProvider collisions → unique rows per invocation + `ThreadLocal`; Excel reader slow → cache file read once. Anti: hardcoded data in tests; non-parallel provider for 1000 rows (slow); shared mutable rows. Qs: DataProvider vs `@Parameters`? How do you parallelize safely?

30.10 TestNG Listener — Refined

Theory: `ITestListener` hooks into TestNG lifecycle to centralize reporting, screenshots, and logging without cluttering tests. Override `onTestFailure` / `onTestSuccess` / `onTestSkipped` to capture evidence and push to ExtentReports. Register via `@Listeners` or `testng.xml`. For Deloitte frameworks, listeners enable audit trails and flaky-test diagnostics required in regulated projects. Enterprise Relevance: Listeners separate test intent from infrastructure (evidence, reporting, Jira) — tests stay readable, compliance stays automatic.

```
public class TestListener implements ITestListener {
    @Override
    public void onTestFailure(ITestResult r) {
        WebDriver d = DriverManager.get();
        String shot = ScreenshotUtil.capture(d, r.getName());
        ExtentManager.getTest().fail(r.getThrowable(),
        MediaEntityBuilder.createScreenCaptureFromPath(shot).build());
    }
    @Override public void onTestSuccess(ITestResult r) {
        ExtentManager.getTest().pass("PASS"); }
}
```

Triage: listener NPE (driver null on config failure) → null-guard; Extent thread-mixing parallel → `ThreadLocal`. Anti: logic in listeners (belongs in utils); swallowing listener exceptions (hides evidence

gaps). Qs: Listener vs BaseTest teardown — what lives where?

30.11 Retry Analyzer — Refined

Theory: `IRetryAnalyzer` reruns transiently failed tests up to `maxRetryCount`, reducing CI noise from flaky UI/API timing. Implement `retry()` with counter (cap 1-2); attach via `@Test(retryAnalyzer=...)` or `IAnnotationTransformer` globally. Log retries distinctly so reports distinguish real defects from infra flakiness. Never retry assertion failures blindly — cap prevents masking. Enterprise Relevance: Retry + quarantine + flake dashboard form the flake-management trio (Sec 29). Retry buys signal (pass-on-retry = flaky, not failed); cap + logging prevent masking.

```
public class Retry implements IRetryAnalyzer {
    private int count = 0; private static final int MAX = 2;
    @Override public boolean retry(ITestResult r) {
        if (count++ < MAX) { Log.warn("Retry {} attempt {}", r.getName(), count); return
true; }
        return false;
    }
}
// global: Transformer sets retryAnalyzer when absent (see 17.39 pattern)
```

Triage: all-green via retry → masking (check flake bucket); non-idempotent POST retried → duplicates (gate retry to idempotent/flaky-only). Anti: infinite retries; retrying asserts; no retry logging. Qs: Retry vs quarantine vs fail?

30.12 Config Reader — Refined

Theory: Externalizes environment data — URLs, timeouts, browser, credentials — from code into `config.properties` or YAML. Singleton `ConfigReader` loads `Properties` once, exposes typed `get()` with `-D` system-property override (same artifact across DEV/QA/UAT via `-Denv=qa`). Never hardcode secrets; resolve from env vars or vault in enterprise frameworks. Enterprise Relevance: Config reader is what makes "same binary, every env" possible (Sec 21.3). Typed getters + fail-fast on missing keys prevent silent misconfiguration.

```
public final class ConfigReader {
    private static final Properties P = new Properties();
    static {
        String env = System.getProperty("env", "qa");
        try (InputStream is = Files.newInputStream(Path.of("src/test/resources/" + env +
".properties"))) {
            P.load(is);
        } catch (IOException e) { throw new ExceptionInInitializerError("Missing config
for env=" + env); }
    }
    private ConfigReader() {}
    public static String get(String k) { return System.getProperty(k,
P.getProperty(k)); }
    public static int timeoutSecs() { return Integer.parseInt(get("timeout.secs")); }
}
```

Triage: wrong-env run → -Denv check + logged active env; missing key NPE → typed getter with default/require. Anti: hardcoded URLs; secrets in properties (use vault); reload per call (cache once).
Qs: How does one artifact run in 3 envs?

30.13 Test Data Reader — Refined

Theory: Feeds data-driven tests from Excel, CSV, or JSON via TestNG `@DataProvider` or Jackson/Apache POI. Separates test logic from datasets, enabling boundary, negative, and localization coverage. Returns `Object[][]` or `Iterator<Object[]>`; caches file reads; validates schema upfront. Prefer JSON for API payloads (typed, versioned); Excel only when non-technical testers maintain cases. Enterprise Relevance: Data readers turn 1 method into 100 cases. Schema validation on load fails fast on data drift (not 50 confusing test failures).

```
@DataProvider(name = "users")
public Object[][] users() throws Exception {
    String json = Files.readString(Path.of("src/test/resources/data/users.json"));
    User[] arr = new ObjectMapper().readValue(json, User[].class);
    return Arrays.stream(arr).map(u -> new Object[] { u }).toArray(Object[][]::new);
}
```

Triage: data-drift failures → schema validation on load; Excel slowness → JSON migration; encoding issues → UTF-8 explicit. Anti: inline data in tests; Excel for API payloads; no schema check. Qs: JSON vs Excel — when each?

30.14 REST Assured Request Specification — Refined

Theory: `RequestSpecification` built with `RequestSpecBuilder` centralizes base URI, base path, headers, logging, timeouts for reuse. Share via static `spec()` factory across tests to enforce standards and simplify environment switching. Combine with `LogDetail.ALL` on failure only (readable CI logs). This pattern is the API equivalent of Selenium `BasePage` — common setup inherited, test-specific details per call. Enterprise Relevance: Centralized specs make auth/header/timeout changes one-line fixes across 500 API tests. Per-test specs drift and rot.

```
public final class ApiSpecs {
    private ApiSpecs() {}
    public static RequestSpecification spec() {
        return new RequestSpecBuilder()
            .setBaseUri(ConfigReader.get("api.url"))
            .setContentType(ContentType.JSON)
            .addFilter(new AllureRestAssured())
            .log(LogDetail.ALL).build();
    }
    public static RequestSpecification authedSpec() {
        return given().spec(spec()).header("Authorization", "Bearer " + Auth.token());
    }
}
```

Triage: auth failures everywhere → spec token refresh (not per-test); log noise → failure-only logging. Anti: per-test baseURI/headers; logging ALL always (GB logs); hardcoded URLs. Qs: What belongs in shared spec vs per-test?

30.15 REST Assured API Client — Refined

Theory: An API Client wrapper hides RestAssured verbs behind domain methods like `createUser(User)` returning deserialized POJOs. Accepts shared `RequestSpecification`, applies path/query/body, calls `given().spec().when()`, extracts with `.as()`. Yields readable BDD tests, single-point maintenance when contracts change, easy mocking for contract-driven engagements. Enterprise Relevance: Clients are the API equivalent of Page Objects — tests express intent (`users.create(admin)`), never HTTP mechanics. Contract change = 1 client method, not 50 tests.

```
public class UserClient {
    public User create(User u) {
        return given().spec(ApiSpecs.authedSpec()).body(u)
            .when().post("/users")
            .then().statusCode(201).extract().as(User.class);
    }
    public void delete(long id) {
        given().spec(ApiSpecs.authedSpec()).when().delete("/users/{id}", id)
            .then().statusCode(204);
    }
}
```

Triage: contract drift breaks one client method (fix once); tests asserting HTTP details → move to client. Anti: RestAssured DSL in tests; string URLs per test; no POJOs (Map soup). Qs: Client vs spec responsibilities? How do you mock clients?

30.16 Authentication Provider — Refined

Theory: Isolates token acquisition — OAuth2 client-credentials, Basic, API-key, JWT login — from test flow. Fetches, caches until expiry (with buffer), injects `Authorization: Bearer <token>` via header or `auth().oauth2()`. Centralization handles refresh, multi-role users, secret masking. Store client id/secret in env vars, never in Git; scope tokens per test role. Enterprise Relevance: Auth is cross-cutting — centralized provider means rotation/expiry fixes happen once. Per-test auth logic rots into expiry flakes.

```
public final class Auth {
    private static String token; private static Instant expiry;
    private Auth() {}
    public static synchronized String token() {
        if (token == null || Instant.now().isAfter(expiry.minusSeconds(30))) {
            token = given().formParam("grant_type", "client_credentials")
                .auth().preemptive().basic(System.getenv("CLIENT_ID"),
                System.getenv("CLIENT_SECRET"))
                .when().post(System.getenv("AUTH_URL")).then().statusCode(200)
                .extract().path("access_token");
            expiry = Instant.now().plusSeconds(3600);
        }
        return token;
    }
}
```

Triage: 401s everywhere → expiry/rotation (not per-test); parallel token thrash → synchronized + buffer. Anti: hardcoded secrets; per-test login; no expiry handling; tokens in logs. Qs: How do you handle expiry + multi-role + masking centrally?

30.17 Response Validation — Refined

Theory: Asserts status code, headers, JSON schema, and business fields using Hamcrest matchers, `JsonPath`, and POJO equality. Validate `statusCode`, `time(lessThan())`, schema via `matchesJsonSchemaInClasspath()`, then deserialize to assert invariants. Layer strict contract checks (schema, required fields) plus focused functional assertions (business values) to catch breaking API changes early without brittle whole-payload string compares. Enterprise Relevance: Layered validation (protocol → contract → business) pinpoints breakage layer: status fail = down; schema fail = contract drift; value fail = logic bug. One assertion style can't distinguish these.

```
given().spec(ApiSpecs.authedSpec())
  .when().get("/users/{id}", 101)
  .then().statusCode(200)
  .time(lessThan(1500L))
  .header("Content-Type", containsString("json"))
  .body("id", equalTo(101))
  .body("email", containsString("@"))
  .body(matchesJsonSchemaInClasspath("schemas/user-schema.json"));
```

Triage: schema fail → contract drift (version/pact check); value fail → logic; time fail → perf (not functional). Anti: whole-payload string compare (brittle); status-only asserts; no schema validation. Qs: Three validation layers + what each catches?

30.18 Playwright Page Object — Refined

Theory: Encapsulates selectors and actions per page into reusable classes, reducing duplication and maintenance in large suites. Each page exposes `Locator`s in constructor and async methods for navigation/workflows. Tests instantiate objects with `page` fixture — readable, stable vs inline selectors. Prefer `getByRole` / `getByTestId` locators; assertions inside page methods (navigational) stay in tests for business asserts. Enterprise Relevance: POM is the maintainability contract — UI change touches 1 class. Typed locators (`readonly Locator`) catch refactors at compile time.

```
export class PlaywrightDevPage {
  readonly searchBox: Locator;
  constructor(readonly page: Page) {
    this.searchBox = page.getByRole('searchbox', { name: 'Search' });
  }
  async goto() { await this.page.goto('https://playwright.dev'); }
  async search(text: string) { await this.searchBox.fill(text); await
this.searchBox.press('Enter'); }
}
```

Triage: locator drift → 1 class fix; flaky page method → web-first asserts inside. Anti: inline selectors in tests; page methods asserting business outcomes (belongs in test); static shared page. Qs: What lives in POM vs test?

30.19 Playwright Fixture — Refined

Theory: Fixtures provide isolated, auto-setup/teardown test dependencies via `test.extend()`. Built-ins: `page`, `context`, `browser`, `request`; custom fixtures supply Page Objects, test data, DB handles. Declare types; `await use(value)` hands off; cleanup runs after. Worker/test scope, overrides, `auto` mode, composition for scalable frameworks. Enterprise Relevance: Fixtures replace repetitive `beforeEach` login/setup with typed, isolated, composable dependencies — the Playwright equivalent of TestNG providers + ThreadLocal combined.

```
import { test as base } from '@playwright/test';
import { TodoPage } from './todo-page';
type F = { todoPage: TodoPage };
export const test = base.extend<F>({
  todoPage: async ({ page }, use) => {
    const t = new TodoPage(page); await t.goto(); await use(t);
    // teardown here (cleanup test data)
  },
});
export { expect } from '@playwright/test';
```

Triage: shared state across tests → fixture scope wrong (worker vs test); slow setup → worker scope + lazy. Anti: globals instead of fixtures; no teardown; untyped fixtures. Qs: Fixture vs beforeEach? Worker vs test scope?

30.20 Playwright Config — Refined

Theory: `playwright.config.ts` centralizes timeouts, retries, workers, `testDir`, `baseURL`, devices, projects, reporters, traces, video, `webServer` via `defineConfig()`. Shared `use: {}` options globally, overridden per project (Chromium/Firefox/WebKit, mobile, staging/prod, smoke/regression splits). Env vars drive CI parallelism, sharding, `globalTimeout` — deterministic local + pipeline execution. Enterprise Relevance: Config is the control plane — projects encode the entire test matrix (browser × env × suite). One file review covers cross-browser, sharding, artifacts, and gates.

```
export default defineConfig({
  testDir: 'tests', fullyParallel: true, workers: process.env.CI ? 2 : 4,
  retries: process.env.CI ? 2 : 0, reporter: [['html', { open: 'never' }]],
  ['blob']],
  use: { baseURL: 'http://localhost:3000', trace: 'on-first-retry', screenshot:
    'only-on-failure' },
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox', use: { ...devices['Desktop Firefox'] } },
  ],
  webServer: { command: 'npm start', url: 'http://localhost:3000',
    reuseExistingServer: !process.env.CI },
});
```

Triage: flaky CI timeouts → per-project timeouts; missing baseURL → relative goto fails; webServer port clash → reuseExistingServer. Anti: hardcoded URLs in tests; no projects (single browser); trace/video always-on. Qs: `use` vs project-level overrides? What must differ per project?

30.21 Playwright Authentication State — Refined

Theory: Playwright reuses login via `storageState` (cookies, localStorage, IndexedDB, passkeys persisted to JSON). Recommended pattern: authenticate once in `auth.setup.ts`, save under `playwright/.auth/`, load through project dependencies (`dependencies: [ 'setup' ]`). Avoids per-test logins (minutes saved), accelerates suites, supports multiple roles/workers (separate files per role). Never commit auth files (gitignore); refresh on expiry; isolate browser-specific sessions. Enterprise Relevance: Auth-once is the biggest Playwright speed lever (login 10s × 200 tests = 33min saved). Per-role files enable RBAC testing without login duplication.

```
// auth.setup.ts – runs once, all projects depend on it
import { test as setup } from '@playwright/test';
setup('authenticate', async ({ page, context }) => {
  await page.goto('/login');
  await page.getByLabel('Password').fill(process.env.PW!);
  await page.getByRole('button', { name: 'Sign in' }).click();
  await page.waitForURL('/');
  await context.storageState({ path: 'playwright/.auth/user.json' });
});
// config: projects: [{ name: 'setup', testMatch: /\.*\setup\.ts/ }, { name:
'chromium', dependencies: ['setup'], use: { storageState:
'playwright/.auth/user.json' } }]
```

Triage: expired state → scheduled refresh + setup-project health check; cross-role pollution → separate files; committed secrets → rotate + gitignore + purge. Anti: login per test; committed `.auth/`; shared single state parallel (overwrite races). Qs: Setup project + dependencies — how does reuse work?

30.22 Playwright API Client — Refined

Theory: `APIRequestContext` enables direct REST testing, setup, and postcondition validation without browser overhead. Built-in `request` fixture honors `baseURL` and headers; or create isolated contexts with `playwright.request.newContext()`. Supports GET/POST/PUT/DELETE, JSON/form/multipart, shared browser cookies via `page.request`. Ideal for seeding microservices, creating test entities, verifying backend persistence — hybrid API+UI in one tool. Enterprise Relevance: One tool for setup (API, ms) + verification (UI) + assertion (API/DB) eliminates RestAssured+Selenium dual-stack complexity for Playwright shops.

```
test('create issue', async ({ request, page }) => {
  const r = await request.post('/repos/u/repo/issues', { data: { title: 'bug' } });
  expect(r.ok()).toBeTruthy();
  const { id } = await r.json();
  await page.goto(`/issues/${id}`); // UI asserts on API-created reality
  await expect(page.getByTestId('title')).toHaveText('bug');
});
```

Triage: shared browser cookies needed → `page.request` (inherits storage); isolated service tests → `request.newContext` + `dispose`. Anti: UI clicks for API-creatable setup; no disposal (socket leaks); asserting API logic via UI (slow). Qs: `request` fixture vs `newContext` vs `page.request` ?

30.23 CI Pipeline — Refined

Theory: Playwright runs on GitHub Actions, Jenkins, GitLab, CircleCI, Azure, Docker: `npm ci` → `npx playwright install --with-deps` → `npx playwright test`. Standard workflow triggers on push/PR, uploads HTML reports/traces, configures sharding, retries, `globalTimeout`. Multi-stage pipelines separate lint, smoke, E2E, API, security scans with environment-specific projects and artifacts for reliable continuous validation. Enterprise Relevance: Standardized pipeline (lint → smoke → E2E → report) across repos means any engineer can debug any pipeline. Sharding + blob merge is the scale pattern.

```
- run: npm ci
- run: npx playwright install --with-deps chromium
- run: npx playwright test --shard=${{ matrix.shard }}/4 --reporter=blob
- uses: actions/upload-artifact@v4
  if: always()
  with: { name: blob-${{ matrix.shard }}, path: blob-report }
```

Triage: browser-install drift → version pin + cache; no sharding (45min runs) → matrix; missing reports → `always()` upload. Anti: `npm install` (nondeterministic); all browsers installed (slow); no `globalTimeout` (hung suite burns agents). Qs: What are the pipeline stages and their budgets?

30.24 SQL Validation Utility — Refined

Theory: SQL utilities extend Playwright for true end-to-end validation by querying PostgreSQL/MySQL/SQL Server after UI/API actions. Typical `dbClient.ts`: connection pools, repository methods, fixtures for seeding/assertions/cleanup. Patterns: transaction rollback isolation, per-worker data, JOIN/integrity checks, audit-trail verification. Helper libraries (e.g., `playwright-db-helper`) simplify setup and prevent cross-test contamination. Enterprise Relevance: UI asserts appearance; DB asserts persistence. The pair closes the loop (Sec 20.18) — green UI + missing row = false pass caught here.

```
import { Pool } from 'pg';
export const db = new Pool({ host: process.env.DB_HOST, database: 'test_db', max: 5 });
export async function orderStatus(email: string) {
  const { rows } = await db.query('SELECT status FROM orders WHERE email = $1', [email]);
  return rows[0]?.status;
}
// test: await expect.poll(() => orderStatus(email)).toBe('PAID');
```

Triage: connection leaks → pool + dispose; cross-test rows → per-worker scoping (Sec 23.11); `async` writes → poll, not sleep. Anti: string-concat SQL (injection even in tests); shared tables; no cleanup. Qs: UI vs DB assertion — what does each prove?

30.25 Logging Utility — Refined

Theory: Logging utilities add structured diagnostics beyond `console` output using Winston, Pino, or custom `Logger` fixtures. Attach per-test loggers via `test.extend`, including `testInfo.title`, status, retry count, timestamps, screenshots. File/JSON transports aid CI debugging while

`testInfo.attach()` embeds logs in reports. Centralize log levels, redaction (secrets/PII), correlation IDs, worker-safe output paths for maintainable failure analysis. Enterprise Relevance: Structured per-test logs turn "it failed" into "order 412 failed at payment step, retry 1, trace attached". Correlation IDs link test logs to app logs to traces.

```
// fixture: per-test logger with test context
logger: async ({}, use, testInfo) => {
  const logger = new Logger({ test: testInfo.title, retry: testInfo.retry, runId:
process.env.RUN_ID });
  await use(logger);
  testInfo.attach('logs', { body: logger.dump(), contentType: 'application/json' });
},
// usage: await logger.info('order created', { orderId });
```

Triage: interleaved parallel logs → per-test files + run/test IDs; secrets in logs → redaction list + review. Anti: `console.log` everywhere; global log file (parallel interleave); PII/tokens logged. Qs: What context must every log line carry?

30.26 Reporting Utility — Refined

Theory: Reporting utilities transform results into actionable HTML dashboards using Playwright HTML, Allure, JUnit, JSON, Blob, or custom reporters. Configure multiple reporters in `playwright.config.ts`; preserve traces/screenshots/videos on failure; publish artifacts in CI. Allure adds steps, labels, environment info, categories, history, and trace links via `allure-playwright`. Standardize retention, naming, and flaky-test visibility for stakeholders. Enterprise Relevance: Reports are the stakeholder interface — managers decide releases from dashboards, not terminals. History + categories + flake visibility turn reports from pass/fail into release intelligence.

```
export default defineConfig({
  reporter: [
    ['html', { open: 'never' }],
    ['junit', { outputFile: 'results/junit.xml' }],
    ['blob', { outputDir: 'blob-report' }],
    ['allure-playwright', { resultsDir: 'allure-results' }],
  ],
});
// categories: product-defect vs env-flake; history: trend across runs
```

Triage: huge artifacts → failure-only retention; history missing → Allure history directory persisted; flake invisible → retries categorized (passed/flaky/failed). Anti: HTML only (no history); no categories (all failures look same); infinite retention. Qs: What turns a report into release intelligence?

30.27 Anti-Patterns & Critical Pitfalls

- **Hardcoding Credentials:** Storing API keys or passwords in code instead of HashiCorp Vault or environment variables.